CFG paper

Ilan Bronstein

September 2026

1 Setup

1.1 Diffusion Models

Diffusion models are generative models that learn to sample from a target distribution $p_{\text{data}}(x_0)$ by sampling noise and reversing a gradual noising process. Given a clean sample $x_0 \sim p_{\text{data}}(x_0)$, the forward diffusion process constructs a continuous-time trajectory $\{x_t\}_{t\in[0,T]}$ governed by the time-varying Ornstein-Uhlenbeck (OU) SDE:

$$dx_t = -\frac{1}{2}\beta_t x_t\, dt + \sqrt{\beta_t}\, dW_t \tag{1}$$

where W_t is a standard Wiener process and $\beta_t > 0$ is a deterministic variance schedule (usually $\beta_t = 2$). Because the OU process is a linear Gaussian SDE, its marginal transition kernel at any time t has a closed-form solution:

$$q(x_t|x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I}\right) \quad \Longleftrightarrow \quad x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0,\mathbf{I}) \tag{2}$$

where $\bar{\alpha}_t = e^{\left(-\int_0^t \beta_s\, ds\right)}$ (or $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ with $\alpha_s = 1-\beta_s$ in discrete time), and $p_t(x_t) = \int q(x_t \mid x_0) p_{\text{data}}(x_0)\, dx_0$, and q is the underlying true distribution of the forward process. As t increases, the OU drift pulls the distribution $p_t(x_t)$ exponentially fast toward its stationary Gaussian prior $\mathcal{N}(0,\mathbf{I})$.

By Anderson's theorem there exists a reverse process governed by the SDE:

$$dx_t = \left[-\frac{1}{2}\beta_t x_t - \beta_t \nabla_{x_t} \log p_t(x_t)\right] dt + \sqrt{\beta_t}\, d\bar{W}_t \tag{3}$$

where $\bar{W}_t$ is a reverse-time Wiener process, alongside a deterministic probability flow ODE: $dx_t = \left[-\frac{1}{2}\beta_t x_t - \frac{1}{2}\beta_t \nabla_{x_t} \log p_t(x_t)\right] dt$.

The marginal distributions of the forward process when starting at $t = 0$, $p_0 = p_{data}$, are the same as the marginals of the reverse process when starting at $t = T$ (or more accurately $t \to \infty$), and $p_\infty = \mathcal{N}(0,\mathbf{I})$. This can be shown by equating the KFE's of each of these processes, where the forward's process is the regular KFE - $\frac{\partial p_t(x)}{\partial t} = -\nabla_{div} \cdot \left(-\frac{1}{2}\beta_t x\, p_t(x)\right) + \frac{1}{2}\beta_t \Delta p_t(x)$,
and the reverse's is the KBE - $\frac{\partial q_\tau(x)}{\partial \tau} = -\nabla_{div} \cdot \left(\left[\frac{1}{2}\beta_{T-\tau}x + \beta_{T-\tau}\nabla_x \log p_{T-\tau}(x)\right] q_\tau(x)\right) +$

$\frac{1}{2}\beta_{T-\tau}\Delta q_\tau(x)$. Where we defined $\tau = T - t$, $q_\tau(x)$, starting from $\tau = 0$ ending at $\tau = T$, the probability density of this reverse process at time τ, and applied it to Eq. (3). We will show that $q_\tau(x) = p_{T-\tau}(x)$.
First the right hand side results in: $\frac{\partial p_{T-\tau}}{\partial \tau} = -\frac{\partial p_t}{\partial t}$. In the left hand side we distribute the divergence operator, and use the identity $p_t \nabla \log p_t = \nabla p_t$ to get: $\frac{\partial q_\tau(x)}{\partial \tau} = -\nabla_{div} \cdot \left(\frac{1}{2}\beta_t x\, p_t\right) - \beta_t \nabla_{div} \cdot (\nabla p_t) + \frac{1}{2}\beta_t \Delta p_t = -\nabla_{div} \cdot \left(\frac{1}{2}\beta_t x\, p_t\right) - \beta_t \Delta p_t + \frac{1}{2}\beta_t \Delta p_t = -\nabla_{div} \cdot \left(\frac{1}{2}\beta_t x\, p_t\right) - \frac{1}{2}\beta_t \Delta p_t$.
Equating the 2 sides results in: $\frac{\partial p_t}{\partial t} = -\nabla_{div} \cdot \left(-\frac{1}{2}\beta_t x\, p_t\right) + \frac{1}{2}\beta_t \Delta p_t$, which is exactly the KFE governing the forward process. We showed that the distribution of the reverse process, starting from $\tau = 0 \Rightarrow t = T \Rightarrow q_0 = \mathcal{N}(0, \mathbf{I})$, and follows this exact process, will reach at $\tau = T$ meaning $t = 0$ the same distribution as $p_0 = p_{data} \Rightarrow q_T = p_{data}$.

The score $\nabla_{x_t} \log p_t(x_t)$ that appears in the reverse process is a vector field that points toward the direction in which the probability density rises the most, while its magnitude indicates how steep the rise is. Its necessity can be understood through Tweedie's formula, which directly relates the score to the optimal denoising of the state. Based on the transition kernel in Eq. (2), the variance of the added noise is $\sigma_t^2 = 1 - \bar{\alpha}_t$, and Tweedie's formula provides the Minimum Mean Square Error (MMSE) estimate of the original data x_0 given the noisy observation x_t:

$$\mathbb{E}[x_0|x_t] = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t + \sigma_t^2 \cdot \nabla_{x_t} \log p_t(x_t)) \tag{4}$$

Because the forward process transition is Gaussian, the marginal distribution $p_t(x_t)$ is an infinite mixture of Gaussians centered around every possible data point in p_{data} - $p_t(x_t) = \int p(x_t|x_0)p(x_0)dx_0$.
Taking the gradient of the log of this mixture yields a weighted average of directions pointing back to the most likely clean data points, weighted by their posterior probabilities:
$\nabla_{x_t} \log p_t(x_t) = \int \left(-\frac{x_t - \alpha_t x_0}{\sigma_t^2}\right) \frac{p(x_t|x_0)p(x_0)}{p_t(x_t)} dx_0 = \int \left(-\frac{x_t}{\sigma_t^2} + \frac{\alpha_t x_0}{\sigma_t^2}\right) p(x_0|x_t)dx_0 = -\frac{x_t}{\sigma_t^2} + \frac{\alpha_t}{\sigma_t^2}\mathbb{E}[x_0|x_t]$. Meaning the score tells us where we need to go, based on our current location x_t, in order to reach the highest likelihood point x_0. While the Brownian motions dW_t, $d\bar{W}_t$ inherently diffuses and spreads probability mass, the score term injects this optimal MMSE estimate into the reverse process, and provides a force that counteracts the entropy of the noise and guides the trajectories back into p_{data}.

1.2 Training

While generating samples via the reverse process requires the marginal score $\nabla_{x_t} \log p_t(x_t)$ or, in discrete time, the true reverse transitions $q(x_{t-1} \mid x_t)$, both are intractable because they depend on integrating over the unknown data distribution $q(x_0) = p_{data}(x_0)$. To overcome this, we approximate the reverse transitions as Gaussian distributions $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 \mathbf{I})$ with fixed variance $\sigma_t^2 \mathbf{I}$, and train the network $\mu_\theta(x_t, t)$ by minimizing the Variational Lower Bound (ELBO) on the negative log-likelihood. Even though

the marginal reverse transition $q(x_{t-1}|x_t)$ is intractable, conditioning on the clean sample x_0 via Bayes' rule yields a tractable forward process posterior $q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)} = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t \mathbf{I})$ because all three distributions are known forward Gaussians, where $\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$. Decomposing the ELBO into step-wise KL divergences $KL\left(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t)\right)$ between these two Gaussians reduces the training objective at step t to an L_2 regression loss that trains $\mu_\theta(x_t, t)$ to match the closed-form posterior mean $\tilde{\mu}_t(x_t, x_0)$:

$$L_{t-1}(\theta) = \mathbb{E}_{x_0, x_t \sim q}\left[\frac{1}{2\sigma_t^2}||\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)||_2^2\right] + C, \tag{5}$$

where $\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$. Additionally, because the forward marginal $q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I})$ can be reparameterized as $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ with $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, the target posterior mean $\tilde{\mu}_t(x_t, x_0)$ can be rewritten purely as a function of x_t and the injected noise ϵ. Consequently, parameterizing the model to predict the posterior mean $\mu_\theta(x_t, t)$ is mathematically equivalent—up to a time-dependent weighting factor—to training a noise-prediction network $\epsilon_\theta(x_t, t)$ via the simplified Denoising Score Matching objective:

$$L_{DSM}(\theta) = \mathbb{E}_{t, x_0, \epsilon}\left[||\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t\right)||_2^2\right] \tag{6}$$

At the optimum, posterior mean prediction, noise prediction, and score estimation are unified: both $\mu_\theta(x_t, t)$ and $\epsilon_\theta(x_t, t)$ implicitly recover the intractable marginal score function. If we take the conditional Gaussian from before $q(x_t|x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I}\right)$, from Eq. (2) we get $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$. Then, if we take the gradient of the log of this conditional Gaussian with respect to x_t, we get:$\nabla_{x_t} \log q(x_t|x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1-\bar{\alpha}_t}$. After using the x_t definition together with training over all the data x_0, we achieve the marginal score with:

$$\nabla_{x_t} \log p_t(x_t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}} \tag{7}$$

allowing us to express downstream guidance mechanisms interchangeably in terms of scores, noise predictions, or posterior means.

1.3 Inference Sampling

At inference time, generating a sample $x_0 \sim p_\theta(x_0)$ proceeds iteratively from pure Gaussian noise $x_T \sim \mathcal{N}(0, \mathbf{I})$ from $t = T$ down to $t = 1$. Because the true clean data x_0 is unavailable during sampling, each reverse transition relies on a one-step denoising estimate $\hat{x}_0(x_t, t) \approx \mathbb{E}[x_0 \mid x_t]$, obtained by inverting the forward reparameterization in Eq. (2) (equivalently, applying Tweedie's formula to the learned score):

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \approx \frac{x_t + (1-\bar{\alpha}_t)\nabla_{x_t} \log p_t(x_t)}{\sqrt{\bar{\alpha}_t}} \tag{8}$$

Substituting $x_0 \leftarrow \hat{x}_0(x_t, t)$ into the tractable posterior mean $\tilde{\mu}_t(x_t, x_0)$ from Eq. (5) yields the parameterized reverse mean $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)$. Now, both stochastic sampling (DDPM, which discretizes the reverse SDE in Eq. (3)) and deterministic sampling (DDIM, which discretizes the probability flow ODE) can be unified under a single update rule from t to $t-1$:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\hat{x}_0(x_t, t)}_{\text{predicted } x_0} + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \underbrace{\epsilon_\theta(x_t, t)}_{\text{direction pointing to } x_t} + \sigma_t z, \quad z \sim \mathcal{N}(0, \mathbf{I}) \tag{9}$$

where the hyperparameter $\sigma_t = \eta\sqrt{\tilde{\beta}_t}$ controls the stochasticity of the trajectory: setting $\eta = 1$ recovers the Markovian DDPM sampler, whereas $\eta = 0$ removes the injected noise entirely, yielding a deterministic mapping from the initial latent x_T to x_0. Each step can be understood as predicting a denoised image $\hat{x}_0$, and noising it in the same direction as x_t but with a slightly smaller noise scale.

2 CFG

2.1 Conditional Sampling

In conditional generation, the goal is to sample from a conditional data distribution $p_{\text{data}}(x_0|c)$ given a conditioning signal c (usually class label or text prompt). Because the forward noising kernel $q(x_t \mid x_0)$ is Markovian and independent of c it doesn't change, while the reverse process simply requires replacing the unconditional marginal score $\nabla_{x_t} \log p_t(x_t)$ with the conditional score $\nabla_{x_t} \log p_t(x_t|c)$. By Bayes' rule, $p_t(x_t|c) = \frac{p_t(c|x_t)p_t(x_t)}{p(c)} \propto p_t(c|x_t)p_t(x_t)$, as $p(c)$ is independent of the diffusion model and is not being learned. This decomposes the conditional score into an unconditional prior term and a likelihood gradient:

$$\nabla_{x_t} \log p_t(x_t|c) = \nabla_{x_t} \log p_t(x_t) + \nabla_{x_t} \log p_t(c|x_t) \tag{10}$$

However we do not have $p_t(c|x_t)$. In previous work it has been tried to learn an auxiliary classifier $p_t(c|x_t)$, the score of which could be substituted in Eq. (10). Weighing said score with a hyperparameter $w > 1$ (the original paper substituted $\mathrm{w} = w + 1$, thus defining $\mathrm{w} > 0$), and using the relationship of Eq. (7) would result in the new noise estimate used in the guided sampling process:

$$\tilde{\epsilon}_\theta(x_t, t, c) = \epsilon_\theta(x_t, t) - w\sqrt{1 - \bar{\alpha}_t}\, \nabla_{x_t} \log p_t(c|x_t) \tag{11}$$

2.2 Implicit classifier

CFG tries to learn the classifier's score direction without training or using an auxiliary classifier. Assume there exists an Implicit classifier $p_t^{imp}(c|x_t)$, using Bayes' rule we get $p_t^{imp}(c|x_t) \propto \frac{p_t(x_t|c)}{p_t(x_t)}$, where $p_t(x_t) = p_t^{imp}(x_t)$ as they are

independent of c. The score of this implicit classifier would be:

$$\begin{aligned}\nabla_{x_t}\log p_t^{imp}(c|x_t) &= \nabla_{x_t}\log p_t(x_t|c) - \nabla_{x_t}\log p_t(x_t)\\ &\approx -\frac{1}{\sqrt{1-\bar{\alpha}_t}}(\epsilon_\theta(x_t,t,c)-\epsilon_\theta(x_t,t,\emptyset))\end{aligned} \tag{12}$$

Altogether, if we want to learn the implicit classifier's score all we need is to learn $\nabla_{x_t}\log p_t(x_t|c)$, and then we can combine Eq (11) with Eq. (12) in addition with the relation of Eq. (7) to get:

$$\tilde{\epsilon}_\theta^{imp}(x_t,t,c) = \epsilon_\theta(x_t,t,\emptyset) + w\cdot(\epsilon_\theta(x_t,t,c)-\epsilon_\theta(x_t,t,\emptyset)) \tag{13}$$

where $\epsilon_\theta(x_t,t,\emptyset) = \epsilon_\theta(x_t,t)$. The paper which introduced CFG trained the diffusion model to predict $\epsilon_\theta(x_t,t,c)$ the conditional noise corresponding to $\nabla_{x_t}\log p_t(x_t|c)$, with the same loss as Eq. (6). There the dataset included (x_0,c) pairs, so they were sampled jointly. c was set to $\emptyset$ with some probability p^{uncond} so that the model would learn both conditional and unconditional noises. The training of an auxiliary classifier was replaced by conditional and unconditional training within the diffusion model itself. At inference, sampling $\tilde{\epsilon}_\theta^{imp}(x_t,t,c)$ replaced $\epsilon_\theta(x_t,t)$ in Eq. (9), resulting in:

$$\begin{aligned}x_{t-1} &= \sqrt{\bar{\alpha}_{t-1}}\frac{x_t-\sqrt{1-\bar{\alpha}_t}\tilde{\epsilon}_\theta^{imp}(x_t,t,c)}{\sqrt{\bar{\alpha}_t}} + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\cdot\tilde{\epsilon}_\theta^{imp}(x_t,t,c)+\sigma_t z\\ x_{t-1} &= \sqrt{\bar{\alpha}_{t-1}}\tilde{x}_0(x_t,t,c) + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\cdot\tilde{\epsilon}_\theta^{imp}(x_t,t,c)+\sigma_t z\end{aligned} \tag{14}$$

From now on we will use $\tilde{x}_0(x_t,t,c)$ for when we use $\tilde{\epsilon}_\theta^{imp}(x_t,t,c)$ in Eq. (8).

3 SDS

The DSM loss in Eq (6) compares the true noise ϵ with the predicted noise at a noisy image $\epsilon_\theta(x_t,t) = \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0+\sqrt{1-\bar{\alpha}_t}\epsilon,t)$ in order to take a gradient with respect to θ, and change the model's weights. We can, however, also minimize said loss with respect to x_0, meaning take a gradient with respect to the highlighted x_0, and thus find the direction in which we should change the sample for it to adhere to the model's predictions, which for a trained frozen model, is the manifold of real-looking images. Explicitly, we want to minimize the following objective:

$$L_{SDS}(x_0) = \mathbb{E}_{t,\epsilon}\left[||\epsilon-\epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0+\sqrt{1-\bar{\alpha}_t}\epsilon,t\right)||_2^2\right] \tag{15}$$

Let's explicitly calculate said gradient:

$\nabla_{x_0}||\epsilon-\epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0+\sqrt{1-\bar{\alpha}_t}\epsilon,t\right)||_2^2 = \frac{\partial||\epsilon-\epsilon_\theta(x_t,t)||_2^2}{\partial\epsilon_\theta(x_t,t)}\frac{\partial\epsilon_\theta(x_t,t)}{\partial x_t}\frac{\partial x_t}{\partial x_0} =$

$$2(\epsilon_\theta(x_t,t)-\epsilon)\cdot\underbrace{\frac{\partial\epsilon_\theta(x_t,t)}{\partial x_t}}_{\text{NN Jacobian}}\cdot\sqrt{\bar{\alpha}_t}.$$

Algorithm 1 Score Distillation Sampling (SDS)

Require: $\boldsymbol{x}_T \sim \mathcal{N}(0, \mathbf{I}_d)$, learning rate $\eta > 0$

1: $\boldsymbol{x}'_0 \leftarrow \boldsymbol{x}_T$
2: **for** $i = 1$ **to** N **do**
3: $\quad t \sim \mathcal{U}(0, T), \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}_d)$
4: $\quad \boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\boldsymbol{x}'_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$
5: $\quad \boldsymbol{x}'_0 \leftarrow \boldsymbol{x}'_0 - \eta w(t)(\epsilon_\theta(\boldsymbol{x}_t, t, c) - \boldsymbol{\epsilon})$ ▷ Update via Eq. (16) using x_t from step 4
6: **end for**
7: **return** $\boldsymbol{x}'_0$

Defining $w(t) = 2\sqrt{\bar{\alpha}_t}$, and dropping the Jacobian into a simple identity matrix $\mathbf{I}$ for computational stability results in:

$$\nabla_{x_0} L_{SDS} \approx \mathbb{E}_{t,\epsilon}[w(t)(\epsilon_\theta(x_t, t) - \epsilon)] \tag{16}$$

This formulation is called Score Distillation Sampling (SDS) as it uses the diffusion model's scores to distill the more likely samples. It is important to note that the SDS loss can be achieved from a weighted probability density distillation loss - $KL(q(x_t|x)||p_\theta(x_t))$, where q is the true conditional distribution and p_θ is the distribution distilled by our diffusion model. With this formulation there is no need to dismiss any Jacobian.
$KL(q(x_t|x)||p_\theta(x_t|c)) = \mathbb{E}_\epsilon[log(q(x_t|x)) - log(p_\theta(x_t|c))]$
$\Rightarrow \nabla_\theta KL(q(x_t|x)||p_\theta(x_t|c)) = \mathbb{E}_\epsilon[\nabla_\theta log(q(x_t|x)) - \nabla_\theta log(p_\theta(x_t|c))]$.
Now the first gradient evaluates to:
$\nabla_\theta log(q(x_t|x)) = (\frac{\partial log(q(x_t|x))}{\partial x} + \frac{\partial log(q(x_t|x))}{\partial x_t} \cdot \frac{\partial x_t}{\partial x})\alpha_t \frac{\partial x}{\partial \theta} = (\frac{\alpha_t}{\sigma_t}\epsilon - \frac{\alpha_t}{\sigma_t}\epsilon)\frac{\partial x}{\partial \theta} = 0$
While the second gradient can be represented with the learned noise ϵ_θ:
$\nabla_\theta log(p_\theta(x_t|c)) = \nabla_{x_t} log(p_\theta(x_t|c))\frac{\partial x_t}{\partial \theta} = -\frac{\alpha_t}{\sigma_t}\epsilon_\theta(x_t, t, c)\frac{\partial x}{\partial \theta}$
Where the last transition used Eq. (4). Eliminating the term $\frac{\partial log(q(x_t|x))}{\partial x} = \frac{\alpha_t}{\sigma_t}\epsilon\frac{\partial x}{\partial \theta}$ which has expectation 0, and thus was shown to lead to reduced variance. Now the remaining terms correspond to the real and predicted noises thus resulting in:
$\nabla_\theta L_{SDS} = \mathbb{E}_{\epsilon, x_t|x}[\nabla_\theta KL(q(x_t|x)||p_\theta(x_t|c))] = \mathbb{E}_{\epsilon,t}[w(t) \cdot (\epsilon_\theta(x_t, t, c) - \epsilon)\frac{\partial x}{\partial \theta}]$.
In fact the KL formulation is the more correct one, but in order to see the different approach (weights vs. images), we used the L_2 formulation. The SDS loss points our current image to the model's ideal denoised image. Of course as we have discussed before we can use guided noise predictions in this approach as well. We can simply plug in $\tilde{\epsilon}_\theta^{imp}(x_t, t, c)$ from CFG to push our image towards a prompt aligned, and realistic looking image. The SDS formulation can be used to sample novel images using algorithm 1.

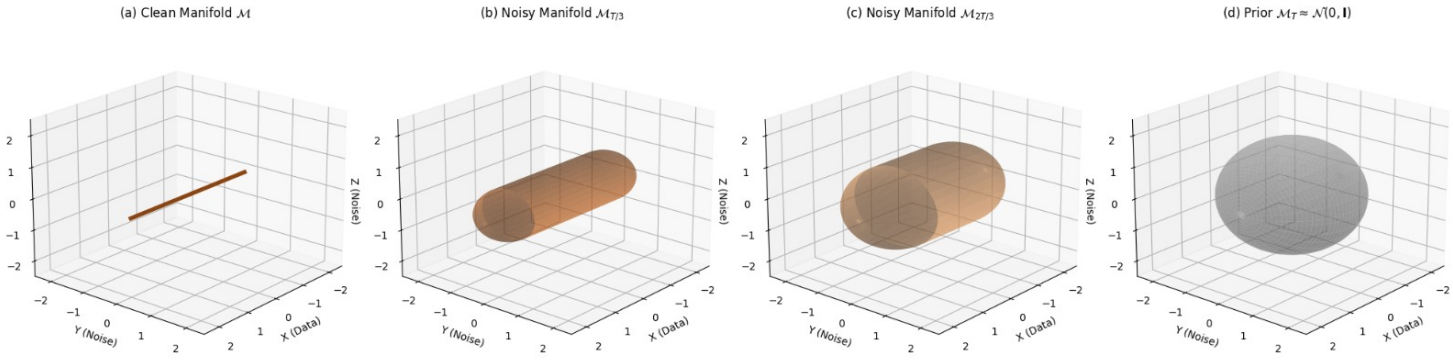


Figure 1: Transition from $\mathcal{M}$ through $\mathcal{M}_{T/3}$ and $\mathcal{M}_{2T/3}$ to $\mathcal{M}_T = N(0, \mathbf{I})$. The data manifold is the 1-dimensional line $y, z = 0$.

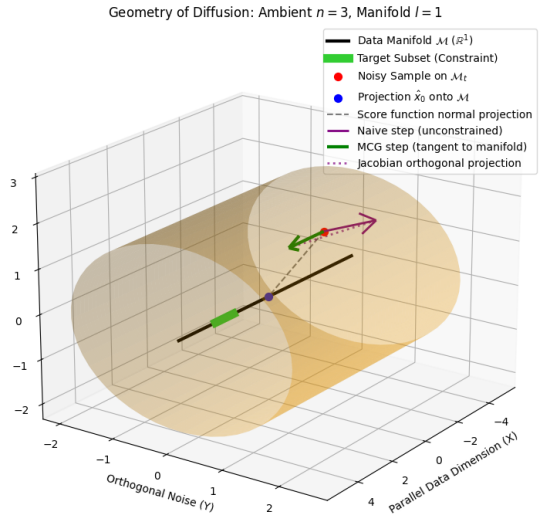


Figure 2: MCG correction onto the constrained subset of $\mathcal{M}$. The data manifold is the 1-dimensional line $y, z = 0$. The MCG step forces the naive gradient step in purple to be parallel to data manifold. The gradient vector isn't an explicit calculation of a specific loss; it was artificially added to show how it must be tangent to the data manifold.

4 MCG

To understand why the aforementioned guidance algorithms introduce artifacts and noise when used, and how to correct them, we need to adopt a geometric view of diffusion models and data manifolds. Under the manifold hypothesis, clean data points $x_0 \sim p_{data}(x_0)$ do not populate the whole pixel space $\mathbb{R}^d$ uniformly, but are supported on a low-dimensional data manifold $\mathcal{M} \subset \mathbb{R}^d$ of intrinsic dimension $l \ll d$. To make the geometric analysis tractable, we assume that $\mathcal{M}$ is locally linear: in a local neighbourhood of a real sample $x_0 \in \mathcal{M}$, the manifold coincides with its l-dimensional tangent space, $\mathcal{M} \cap B_R(x_0) = T_{x_0}\mathcal{M} \cap B_R(x_0)$ with $T_{x_0}\mathcal{M} \cong \mathbb{R}^l$, and p_{data} is locally uniform on $\mathcal{M}$. Where $B_R(x_0)$ is the ball of radius R around x_0, and $T_{x_0}\mathcal{M}$ is the tangent space to $\mathcal{M}$ at point x_0. Thus the forward's noising process of Eq. (2) - ϵ, can be decomposed to an l-dimensional component $\epsilon_{||} \in T_{x_0}\mathcal{M}$ tangent to the manifold and a $(d-l)$-dimensional component $\epsilon_\perp$ orthogonal to the manifold. Because $\mathcal{M}$ is locally flat, the tangent perturbation $\sqrt{1-\bar{\alpha}_t}\epsilon_{||}$ translates the sample along the scaled manifold $\sqrt{\bar{\alpha}_t}\mathcal{M}$ without changing its distance from $\sqrt{\bar{\alpha}_t}\mathcal{M}$. This means the Euclidean distance $d(x_t, \sqrt{\bar{\alpha}_t}\mathcal{M}) := \inf_{y \in \sqrt{\bar{\alpha}_t}\mathcal{M}} ||x_t - y||_2$ is determined entirely by the $(d-l)$ orthogonal noise, $d(x_t, \sqrt{\bar{\alpha}_t}\mathcal{M}) = \sqrt{1-\bar{\alpha}_t}||\epsilon_\perp||_2$. Since $||\epsilon_\perp||_2^2 \sim \chi^2_{d-l}$ is a sum of $d-l \gg 1$ independent squared standard normal variables, high-dimensional concentration of measure implies that $||\epsilon_\perp||_2$ concentrates tightly around $\sqrt{d-l}$. Thus $p_t(x_t)$ is concentrated on a thin $(d-1)$-dimensional noisy manifold $\mathcal{M}_t$ at a fixed radius r_t around the scaled data manifold:
$\mathcal{M}_t := \left\{ y \in \mathbb{R}^d \mid d(y, \sqrt{\bar{\alpha}_t}\mathcal{M}) = r_t := \sqrt{1-\bar{\alpha}_t}\sqrt{d-l} \right\}$.
$\mathcal{M}_t$ can be understood as an interpolation between the gaussian sphere $\mathcal{N}(0, \mathbf{I})$, and the true data manifold $\mathcal{M}$. The diffusion processes simply transports samples from one to another either forwards from $\mathcal{M}$ to $\mathcal{N}(0, \mathbf{I})$, or backwards from $\mathcal{N}(0, \mathbf{I})$ to $\mathcal{M}$. Both forward and unconditional backward diffusion processes transition between adjacent manifolds $\mathcal{M}_t$ and $\mathcal{M}_{t-1}$, an illustration of which can be found in figure 1.

However, these transitions are disrupted when we constrain or guide the generation process. That is because we are trying to land on a specific subset of the manifold $\mathcal{M}$ which adheres to the constraint/condition. These issues lead us to formulate a new target: $\min_{x \in \mathcal{M}} L(x)$, where $L(\cdot)$ is a general loss. The flaw in naive guidance is how this loss is minimized. If we evaluate the gradient of our condition directly at the noisy sample, $\nabla_{x_t} L(x_t) \in \mathbb{R}^d$, the resulting vector lies in the full unconstrained space $\mathbb{R}^d$. It contains orthogonal components that do not align with the tangent space of the data manifold $\nabla_{x_t} L(x_t) \notin T_{x_0}\mathcal{M}$. Then applying this gradient, $x'_t = x_t - \gamma_t \cdot \nabla_{x_t} L(x_t) = (x_{t_{||}} - \gamma_t \cdot \nabla_{x_t} L(x_t)_{||}) + (x_{t_\perp} - \gamma_t \cdot \nabla_{x_t} L(x_t)_\perp)$, provides us with a manifold parallel part, which doesn't change our distance from the manifold $\mathcal{M}$, and with an orthogonal part, the norm of which isn't guaranteed to be $||(x_{t_\perp} - \gamma_t \cdot \nabla_{x_t} L(x_t)_\perp)|| \neq r_t$. That is because $||x_{t_\perp}|| = ||\epsilon_{t_\perp}|| = r_t$, but the gradient and it's norm are unconstrained. This pushes the inference path off the interpolating manifolds $\mathcal{M}_t$ (we showed that $x'_t \notin \mathcal{M}_t$).

If a guided sample was pushed off the manifolds, it might be the reason for the artifacts, as the model never trained on off-manifold samples.

To correct this, we must ensure that our conditional gradient updates respect the geometry of $\mathcal{M}$. We can achieve this by utilizing the learned denoising prediction $\hat{x}_0(x_t, t)$ from Eq. (8), which can be understood as a natural unconditional projection onto $\mathcal{M}$. Define the Tweedie map $Q_t(x_t) := \hat{x}_0(x_t, t, \emptyset) \in \mathcal{M}$. Under an optimal score network, its Jacobian matrix $J_{Q_t} = \frac{\partial \hat{x}_0(x_t,t,\emptyset)}{\partial x_t}$ is symmetric and idempotent ($J_{Q_t}^2 = J_{Q_t} = J_{Q_t}^T$), meaning $J_{Q_t} : \mathbb{R}^d \to T_{\hat{x}_0}\mathcal{M}$ acts locally as an orthogonal projection onto the tangent space $T_{\hat{x}_0}\mathcal{M}$. By passing the loss $L(\cdot)$ through this Tweedie projection, we can force the guidance gradient to inhabit the tangent space strictly. This method, known as Manifold Constrained Gradients (MCG), evaluates the loss at the projected clean state rather than the noisy state:

$$\nabla_{x_t}^{\text{MCG}} L(x_t) := \nabla_{x_t} L(\hat{x}_0(x_t, t, \emptyset)) = J_{Q_t}^T \nabla_{\hat{x}_0} L(\hat{x}_0(x_t, t, \emptyset)) \in T_{\hat{x}_0(x_t,t,\emptyset)}\mathcal{M} \quad (17)$$

Taking a step from $\hat{x}_0 \in \mathcal{M}$ along a tangent vector in $T_{\hat{x}_0}\mathcal{M}$ keeps the updated point $\hat{x}_0 - \gamma_t \nabla_{x_t} L(\hat{x}_0)$ on $\mathcal{M}$ because $\mathcal{M}$ is locally linear. An illustration of this process can be found in Figure 2. The evaluation of the loss on $\hat{x}_0$, but the calculation of the gradient with respect to x_t utilizes the chain rule to forcefully incorporate the projection of the Jacobian and result in an in-tangent gradient.

Plugging this update step into Eq. (9) with $\sigma_t = 0$ results in what is called Diffusion Posterior Sampling (DPS):

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}(\hat{x}_0(x_t, t, \emptyset) - \gamma_t \nabla_{x_t} L(\hat{x}_0(x_t, t, \emptyset))) + \sqrt{1 - \bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t, \emptyset) \quad (18)$$

where $\gamma_t > 0$ is a step size, and $L(\cdot)$ is a general loss. Under piecewise-linear manifold assumptions Decomposed Diffusion Sampling (DDS) replaces $\nabla_{x_t} L(\hat{x}_0(x_t, t, \emptyset))$ with the gradient $\nabla_{\hat{x}_0} L(\hat{x}_0(x_t, t, \emptyset))$, resulting in the DDS update step:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}(\hat{x}_0(x_t, t, \emptyset) - \gamma_t \nabla_{\hat{x}_0} L(\hat{x}_0(x_t, t, \emptyset))) + \sqrt{1 - \bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t, \emptyset) \quad (19)$$

where we eliminate the Jacobian because of computation stability (Jacobian of a full net is expensive and is prone to explode/vanish). Under the aforementioned assumption this step can be shown to be equivalent to the DPS step. Additionally we will later see that the gradient with respect to $\hat{x}_0$ is already in the manifold tangent manifold $T_{\hat{x}_0(x_t,t,\emptyset)}\mathcal{M}$, for our selected loss. Together with the locally linear assumption this ensures we stay on-manifold. To conclude, we now have a way to minimize some objective L (which we will use in a CFG like manner to guide our samples), while generating more realistic samples than before, as they stay on the manifolds the unconditional model trained on.

5 CFG++

Now we want to use a smart loss in Eq. (19) that when it is minimized under a condition, will result in realistic and condition adherent samples. The CFG++

paper states that the SDS loss (Eq. (15)) is actually ideal for this purpose. This is because we can very easily substitute $\epsilon_\theta(x_t, t, c)$ instead of $\epsilon_\theta(x_t, t)$. Using this loss in DDS results in the following update step:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}(\hat{x}_0(x_t, t, \emptyset) - \gamma_t \nabla_{\hat{x}_0} L_{SDS}(\hat{x}_0(x_t, t, \emptyset))) + \sqrt{1 - \bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t, \emptyset) \tag{20}$$

Substituting $\epsilon = \frac{x_t - \sqrt{\bar{\alpha}_t}x}{\sqrt{1-\bar{\alpha}_t}}$ and and $\epsilon_\theta(x_t, t, c) = \frac{x_t - \sqrt{\bar{\alpha}_t}\hat{x}_0(x_t,t,c)}{\sqrt{1-\bar{\alpha}_t}}$ into the SDS loss from Eq. (15) we get: $L_{SDS}(x) = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}||x - \hat{x}_0(x_t, t, c)||^2$. x_t cancel each other out, and we can express the update as:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}(\hat{x}_0(x_t, t, \emptyset) + \lambda\left(\hat{x}_0(x_t, t, c) - \hat{x}_0(x_t, t, \emptyset)\right)) + \sqrt{1 - \bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t, \emptyset) \tag{21}$$

where $\lambda = 2\frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}\gamma_t$, and in practice constricted to be in $[0, 1]$. Then using the CFG notation of Eq. (13) together with $\hat{x}_0(x_t, t, \emptyset) + \lambda\left(\hat{x}_0(x_t, t, c) - \hat{x}_0(x_t, t, \emptyset)\right) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\tilde{\epsilon}_\theta^{imp}(x_t,t,c)}{\sqrt{\bar{\alpha}_t}}$ we get that the update in (21) can be represented as:

$$\begin{aligned} \tilde{x}_0(x_t, t, c) &= \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\tilde{\epsilon}_\theta^{imp}(x_t, t, c)}{\sqrt{\bar{\alpha}_t}} \\ x_{t-1} &= \sqrt{\bar{\alpha}_{t-1}}\tilde{x}_0(x_t, t, c) + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t, t, \emptyset) \end{aligned} \tag{22}$$

Notice how closely this resembles the update rule in the regular CFG algorithm in Eq. (14). The only difference comes in at the renoising step where we switch $\tilde{\epsilon}_\theta^{imp}(x_t, t, c)$ with $\epsilon_\theta(x_t, t, \emptyset)$. This change which again resulted from the use of MCG and DDS forces the next, less noisy image x_{t-1} to still be on the manifold $\mathcal{M}_{t-1}$, and thus achieves better results. More explicitly we can see that $\nabla_{\hat{x}_0} L_{SDS}(\hat{x}_0(\emptyset)) \propto \hat{x}_0(\emptyset) - \hat{x}_0(c)$, which is the difference of two points on the locally linear manifold $\mathcal{M}$, this vector already lies in $T_{\hat{x}_0}\mathcal{M}$.

$\lambda \in [0, 1]$ is therefore chosen so that the next point x_{t-1} will for sure be on-manifold, as it is an interpolation of $\hat{x}_0(\emptyset) - \hat{x}_0(c)$ which are on the manifold. This is an additional difference from regular CFG, which only restricted $w > 1$ - extrapolation. For $w >> 1$ the local linearity assumption no longer holds which may be the reason why CFG produces worse results, as in standard CFG w is usually quite large.

Intuitively this change can be understood as still predicting the most likely image x_0 that adheres to the condition c, but then noising it back into the manifolds the model learned to predict from, instead of the direction from which we came from. It is true that at the previous step we were on such manifold, so one might think traveling back in the exact same direction should still be favourable, however since we change the noising scale at each step we have no guarantee that we will land on a manifold.

CFG++ shows better results than regular CFG, and provides the basis for more complicated algorithms, like ones that use non-constant step sizes λ or w.

6 Annealing Guidance Scale

The scales/step-sizes $w(t)$, λ affect the convergence to an image which is both realistic and prompt adherent. The scale acts as a tradeoff between fidelity and diversity. Increasing the guidance scale prioritizes faithfulness to the prompt, but can lead to mode collapse and artifacts in images. In previous work these scales were constant ($w(t) = c$) through the sampling process, and some have tried learning $w(t)$ as a function of t. However these did not account for many variables in the diffusion space, such as the differing noise scales of the manifolds. Yehezkel et al introduced a new scheduler to learn the guidance scale such that it will consider these variables in mind. The scheduler uses the norm of $\delta_t = \epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \emptyset)$ at each step. Thus the scheduler turn into a function of $w(t, \delta_t)$.

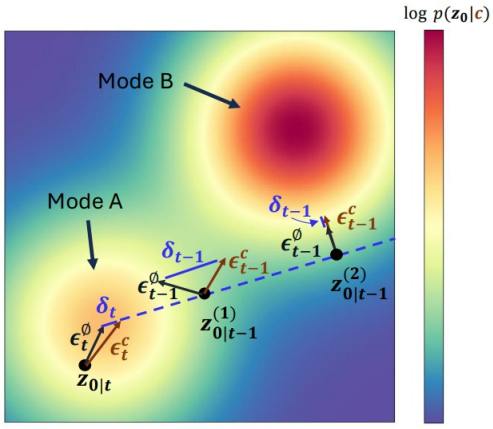


Fig. 4. **Geometric intuition of δ_t.** A 2D illustration showing how the magnitude of $\delta_t = \epsilon^c_t - \epsilon^\emptyset_t$ reflects alignment with the prompt. At time t, the sample $z_{0|t}$ lies near mode A, which partially aligns with the prompt, resulting in a small $\|\delta_t\|$. As the denoising progresses, following the direction of δ_t leads toward mode B that better aligns with the prompt. The candidate points along the line correspond to different guidance scales w. Among these, $z^{(2)}_{0|t-1}$ lies closest to mode B, where the conditional and unconditional predictions are best aligned, yielding a minimal $\|\delta_{t-1}\|$.

Their work was based on the CFG++ framework, which as we discussed, results in δ_t being within the tangent manifold of the data, for all t. Therefore we can focus on the scheduler to learn where the desired probability $p(c|x)$ is highest, because regardless of the scale, it should stay on manifold. Substituting $\delta_t = \epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \emptyset)$ directly into Eq. (16), shows how we can use δ_t as a proxy to minimize the SDS loss. Additionally small $||\delta_t||$ implies agreement between the conditional and unconditional noise, and thus a stationary point of the SDS loss. Intuitively, alignment between the noise predictions implies prompt adherence. This was shown in the original paper in figure 4, which we directly inserted here in Fig. 6

The geometric view suggest we should use $||\delta_{t-1}||$ to train $w(t, \delta_t)$ so that the scheduler will know where the next data point should land to maximize adherence to the prompt.

6.1 The Scheduler

Yehezkel et al propose a light-weight MLP model $w(t, \delta_t, \lambda)$. Where $\lambda \in [0, 1]$ here has nothing to do with the interpolation scale from the CFG++ paper, but is a hyperparameter dictating the fidelity diversity tradeoff. Higher λ emphasizes prompt adherence. At inference the new scheduler replaces w in Eq. (13), and the CFG++ update is usesd.

6.2 Scheduler Training

At training the diffusion model is kept frozen, an image-condition pair (x, c) is sampled together with random timestamp t and noise ϵ. The condition c is perturbed by a gaussian to improve robustness. Then by sampling λ uniformly the CFG noise prediction is achieved (while using the scheduler). Then The loss is:

$$L = \lambda L_{\delta_t} + (1 - \lambda) L_{\epsilon_t} = \lambda ||\delta_{t-1}||^2 + (1 - \lambda) ||\tilde{\epsilon}_t^{imp} - \epsilon||^2 \tag{23}$$

Where we once again reiterate that $\tilde{\epsilon}_t^{imp}$ is calculated with the scheduler. δ_{t-1} is the difference of conditional and unconditional noise predicted for x_{t-1}, which we get by applying the CFG++ update rule for the given x_t (x_t is given by the parameters/data we sampled).
We already discussed how L_{δ_t} acts as a prompt adherence objective, forcing the conditional and unconditional noises of the next steps to align. However minimizing solely this objective results in out of distribution samples, due to extremely high guidance scale. Thus a fidelity objective is in place. L_{ϵ_t} very intuitively forces the predicted noise to match the true sampled noise. This regularization stops the δ loss from pushing the samples to out-of-distribution regions. It is important to note that the guarantee of the CFG++ sampling that all samples will land on the data manifold no longer holds as the scheduler does not restrict the guidance's magnitude, in contrast to CFG++ which forced it to be between 0 and 1. CFG++ uses interpolation thus utilizing the local linearity assumption, which is not guaranteed for larger guidance scales.

The scheduler results in prompt and manifold adherent novel samples, provides differing guidance scales for differing prompts and over-all reduces artifact and leads to more realistic and prompt adherent samples than other SotA schedulers and sampling techniques.

References

[1] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[2] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. DreamFusion: Text-to-3D using 2D diffusion. *arXiv preprint arXiv:2209.14988*, 2022.

[3] Hyungjin Chung, Byeongsu Sim, Dohoon Ryu, and Jong Chul Ye. Improving diffusion models for inverse problems using manifold constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Hyungjin Chung, Jeongsol Kim, Geon Yeong Park, Hyelin Nam, and Jong Chul Ye. CFG++: Manifold-constrained classifier free guidance for diffusion models. *arXiv preprint arXiv:2406.08070*, 2024.

[5] Shai Yehezkel, Omer Dahary, Andrey Voynov, and Daniel Cohen-Or. Navigating with annealing guidance scale in diffusion space. *arXiv preprint arXiv:2408.06079*, 2024.